

Bài tập Mảng một chiều

Nhóm 1: Nhập, Xuất và Tính toán cơ bản

1. `void nhapMang(int a[], int n)`: Viết hàm nhập các phần tử cho mảng.
2. `void xuatMang(int a[], int n)`: Viết hàm xuất các phần tử của mảng.
3. `void phatSinhMangNgauNhien(int a[], int n)`: Viết hàm phát sinh các phần tử ngẫu nhiên cho mảng.
4. `int tinhTongMang(int a[], int n)`: Viết hàm tính tổng các phần tử trong mảng.
5. `long long tinhTichMang(int a[], int n)`: Viết hàm tính tích các phần tử trong mảng.
6. `float tinhTrungBinhCong(int a[], int n)`: Viết hàm tính trung bình cộng của mảng.
7. `int tinhTongPhanTuChan(int a[], int n)`: Viết hàm tính tổng các phần tử chẵn trong mảng.
8. `int tinhTongPhanTuLeViTriLe(int a[], int n)`: Viết hàm tính tổng các phần tử lẻ ở các vị trí lẻ.

Nhóm 2: Tìm kiếm Giá trị và Vị trí

9. `int timMax(int a[], int n)`: Viết hàm tìm phần tử lớn nhất trong mảng.
10. `int timMin(int a[], int n)`: Viết hàm tìm phần tử nhỏ nhất trong mảng.
11. `int timViTriMax(int a[], int n)`: Viết hàm tìm vị trí của phần tử lớn nhất.
12. `int timViTriMin(int a[], int n)`: Viết hàm tìm vị trí của phần tử nhỏ nhất.
13. `int timPhanTuAmLonNhat(int a[], int n)`: Viết hàm tìm phần tử âm lớn nhất. Nếu không có, trả về 0.
14. `int timPhanTuDuongNhoNhat(int a[], int n)`: Viết hàm tìm phần tử dương nhỏ nhất. Nếu không có, trả về 0.
15. `int timViTriDauTien(int a[], int n, int x)`: Viết hàm tìm vị trí xuất hiện đầu tiên của phần tử x . Trả về -1 nếu không tìm thấy.
16. `int timViTriCuoiCung(int a[], int n, int x)`: Viết hàm tìm vị trí xuất hiện cuối cùng của phần tử x . Trả về -1 nếu không tìm thấy.
17. `int timSoChanDauTien(int a[], int n)`: Viết hàm tìm giá trị chẵn đầu tiên trong mảng.
18. `int timSoLeCuoiCung(int a[], int n)`: Viết hàm tìm giá trị lẻ cuối cùng trong mảng.

Nhóm 3: Đếm và Kiểm tra

19. `int demSoNguyenTo(int a[], int n)`: Viết hàm đếm số lượng số nguyên tố trong mảng.
20. `int demSoHoanThien(int a[], int n)`: Viết hàm đếm số lượng số hoàn thiện trong mảng.
21. `int demSoLanXuatHien(int a[], int n, int x)`: Viết hàm đếm số lần xuất hiện của x trong mảng.
22. `int kiemTraMangTangDan(int a[], int n)`: Viết hàm kiểm tra mảng có được sắp xếp tăng dần hay không.
23. `int kiemTraMangDoiXung(int a[], int n)`: Viết hàm kiểm tra mảng có đối xứng hay không.

24. `int kiemTraMangToanChan(int a[], int n)`: Viĩt hàm kiểm tra mảng có toàn phần tử chẵn hay không.

Nhóm 4: Thay đổi Mảng (Thêm, Xóa)

25. `void themPhanTuCuoi(int a[], int &n, int x)`: Viĩt hàm thêm phần tử x vào cuối mảng.
26. `void themPhanTuDau(int a[], int &n, int x)`: Viĩt hàm thêm phần tử x vào đầu mảng.
27. `void themPhanTuViTriK(int a[], int &n, int x, int k)`: Viĩt hàm chèn phần tử x vào vị trí k của mảng.
28. `void xoaPhanTuCuoi(int a[], int &n)`: Viĩt hàm xóa phần tử cuối mảng.
29. `void xoaPhanTuDau(int a[], int &n)`: Viĩt hàm xóa phần tử đầu mảng.
30. `void xoaPhanTuViTriK(int a[], int &n, int k)`: Viĩt hàm xóa phần tử tại vị trí k .
31. `void xoaTatCaPhanTuX(int a[], int &n, int x)`: Viĩt hàm xóa tất cả các phần tử có giá trị bằng x trong mảng.

Nhóm 5: Sắp xếp và Biến đổi

32. `void sapXepTangDan(int a[], int n)`: Viĩt hàm sắp xếp mảng tăng dần (dùng Selection Sort hoặc Bubble Sort).
33. `void sapXepGiamDan(int a[], int n)`: Viĩt hàm sắp xếp mảng giảm dần.
34. `void daoNguocMang(int a[], int n)`: Viĩt hàm đảo ngược thứ tự các phần tử trong mảng.
35. `void dichTraiXoayVong(int a[], int n)`: Viĩt hàm dịch trái xoay vòng mảng 1 đơn vị (phần tử $a[0]$ trở thành $a[n-1]$).
36. `void thayTheXBangY(int a[], int n, int x, int y)`: Viĩt hàm thay thế tất cả các phần tử có giá trị x bằng giá trị y .
37. `void tachMangChanLe(int a[], int n, int mangChan[], int &nChan, int mangLe[], int &nLe)`: Viĩt hàm tách mảng a thành 2 mảng: một mảng chứa số chẵn và một mảng chứa số lẻ.

Nhóm 6: Mảng một chiều nâng cao

38. `void xoaPhanTuTrungNhuu(int a[], int &n)`: Viĩt hàm xóa các phần tử bị trùng trong mảng, chỉ giữ lại phần tử xuất hiện đầu tiên.
39. `void lietKeTanSuat(int a[], int n)`: Viĩt hàm liệt kê tần suất xuất hiện của các phần tử trong mảng. (Gợi ý: Có thể dùng một mảng phụ để đánh dấu các phần tử đã đếm).
40. `void tronHaiMangDaSapXep(int a[], int n, int b[], int m, int c[], int &p)`: Viĩt hàm trộn hai mảng a và b đã được sắp xếp tăng dần thành một mảng c cũng được sắp xếp tăng dần.
41. `void chenMangBVaoMangA(int a[], int &n, int b[], int m, int k)`: Viĩt hàm chèn cả mảng b (gồm m phần tử) vào mảng a (gồm n phần tử) tại vị trí k .
42. `int timDoanConTangDaiNhat(int a[], int n, int &k)`: Viĩt hàm tìm đoạn con (liên tiếp) tăng dài nhất trong mảng. Hàm trả về độ dài của đoạn con và lưu vị trí bắt đầu k .
43. `int timDoanConTongLonNhat(int a[], int n)`: Viĩt hàm tìm đoạn con (liên tiếp) có tổng lớn nhất trong mảng (sử dụng thuật toán Kadane).

44. `void dichPhaiXoayVongKDonVi(int a[], int n, int k)`: Viết hàm dịch phải xoay vòng mảng k đơn vị.
45. `void timCapSoTongBangX(int a[], int n, int X)`: Viết hàm tìm và in ra tất cả các cặp phần tử $(a[i], a[j])$ trong mảng có tổng bằng X (với $i < j$).
46. `void sapXepSoLeTangDan(int a[], int n)`: Viết hàm sắp xếp mảng sao cho các số lẻ tăng dần, các số chẵn giữ nguyên vị trí.
47. `int timPhanTuConThieu(int a[], int n)`: Cho mảng a chứa n số nguyên phân biệt trong khoảng từ 0 đến n . Viết hàm tìm số duy nhất còn thiếu trong khoảng đó. (Ví dụ: mảng $[0, 1, 3, 4]$ với $n = 4$ thì thiếu số 2).

Hướng dẫn

```
#include <stdio.h>

// Hàm hoán vị 2 số nguyên
void hoanVi(int &a, int &b) {
    int t = a; a = b; b = t;
}

// Hàm xóa phần tử tại vị trí k
void xoaPhanTuViTriK(int a[], int &n, int k) {
    if (k < 0 || k >= n)
        return; // Vị trí không hợp lệ
    for (int i = k; i < n - 1; i++)
        a[i] = a[i + 1];
    n--; // Giảm kích thước mảng
}

// Hàm sắp xếp mảng tăng dần
// Sử dụng Bubble Sort cho đơn giản
void sapXepTangDan(int a[], int n) {
    for (int i = 0; i < n - 1; i++)
        for (int j = 0; j < n - i - 1; j++)
            if (a[j] > a[j + 1])
                hoanVi(&a[j], &a[j + 1]);
}

// Hàm in mảng
void xuatMang(int a[], int n) {
    for (int i = 0; i < n; i++)
        printf("%d ", a[i]);
    printf("\n");
}
```

Bài 38: Xóa phần tử trùng nhau

Duyệt mảng. Với mỗi phần tử $a[i]$, duyệt tiếp phần còn lại của mảng (từ $j = i + 1$). Nếu $a[j] == a[i]$, ta gọi hàm xóa phần tử tại vị trí j . Vì mảng bị dồn về sau khi xóa, ta phải giảm j và n .

```
void xoaPhanTuTrungNhuau(int a[], int &n) {
```

```

    for (int i = 0; i < n - 1; i++)
        for (int j = i + 1; j < n; j++)
            if (a[j] == a[i]) {
                xoaPhanTuViTriK(a, n, j);
                j--; // Vì mảng đã dồn lên, ta cần kiểm tra lại vị trí j
            }
}

```

Bài 39: Liệt kê tần suất

Dùng một mảng `danhDau` kích thước n (khởi tạo toàn bộ mảng bằng 0) để đánh dấu các phần tử đã đếm. Duyệt mảng `a`, nếu `danhDau[i] == 0` (chưa đếm), ta tiến hành đếm số lần `a[i]` xuất hiện từ vị trí `i` đến cuối mảng. Khi đếm, nếu `a[j] == a[i]`, ta đặt `danhDau[j] = 1` để không đếm lại.

```

void lietKeTanSuat(int a[], int n) {
    int danhDau[n];
    // Khởi tạo mảng đánh dấu
    for (int i = 0; i < n; i++)
        danhDau[i] = 0;

    for (int i = 0; i < n; i++) {
        if (danhDau[i] == 0) { // Nếu phần tử này chưa được đếm
            int dem = 1;
            for (int j = i + 1; j < n; j++) {
                if (a[j] == a[i]) {
                    dem++;
                    danhDau[j] = 1; // Đánh dấu là đã đếm
                }
            }
            printf("Phan tu %d xuat hien %d lan\n", a[i], dem);
        }
    }
}

```

Bài 40: Trộn hai mảng đã sắp xếp

Dùng 3 chỉ số: `i` cho mảng `a`, `j` cho mảng `b`, `k` cho mảng `c`. So sánh `a[i]` và `b[j]`, phần tử nhỏ hơn sẽ được gán vào `c[k]`. Tăng chỉ số tương ứng (`i` hoặc `j`) và `k`. Lặp lại cho đến khi một trong hai mảng `a` hoặc `b` hết. Cuối cùng, sao chép phần còn lại của mảng kia vào `c`.

```

void tronHaiMangDaSapXep(int a[], int n, int b[], int m, int c[], int &p) {
    int i = 0, j = 0, k = 0;

    // Trộn khi cả hai mảng còn phần tử
    while (i < n && j < m) {
        if (a[i] <= b[j]) {
            c[k] = a[i];
            i++;
        } else {
            c[k] = b[j];
            j++;
        }
        k++;
    }

    // Sao chép phần còn lại của mảng a vào c
    while (i < n)
        c[k++] = a[i++];

    // Sao chép phần còn lại của mảng b vào c
    while (j < m)
        c[k++] = b[j++];
}

```

```

        j++;
    }
    k++;
}

// Sao chép phần còn lại của mảng a (nếu có)
while (i < n) {
    c[k] = a[i];
    i++; k++;
}

// Sao chép phần còn lại của mảng b (nếu có)
while (j < m) {
    c[k] = b[j];
    j++; k++;
}

p = k; // Kích thước của mảng c
}

```

Bài 41: Chèn mảng B vào mảng A

1. Tạo "khoảng trống" trong mảng a bằng cách dời m phần tử, bắt đầu từ cuối mảng a ($a[n-1]$) về $a[k]$. Phải dời từ cuối về để không bị ghi đè.
2. Sao chép các phần tử của mảng b vào khoảng trống vừa tạo (bắt đầu từ $a[k]$).
3. Cập nhật lại kích thước của a (tăng thêm m).

```

void chenMangBVaoMangA(int a[], int &n, int b[], int m, int k) {
    if (k < 0 || k > n)
        k = n; // Nếu k không hợp lệ, chèn vào cuối

    // 1. Tạo khoảng trống
    // Dời các phần tử từ k đến cuối mảng sang phải m vị trí
    for (int i = n - 1; i >= k; i--)
        a[i + m] = a[i];

    // 2. Chép mảng b vào
    for (int i = 0; i < m; i++)
        a[k + i] = b[i];

    // 3. Cập nhật kích thước
    n = n + m;
}

```

Bài 42: Tìm đoạn con tăng dài nhất

Duyệt mảng, dùng 2 biến: `currentLen` (độ dài đoạn tăng hiện tại) và `maxLen` (độ dài lớn nhất tìm được). `currentStart` và `maxStart` để lưu vị trí bắt đầu. Nếu $a[i] > a[i-1]$, tăng `currentLen`. Nếu không, tức là đoạn tăng bị ngắt, ta so sánh `currentLen` với `maxLen`. Nếu `currentLen > maxLen`, cập nhật `maxLen = currentLen` và `maxStart = currentStart`. Reset `currentLen = 1` và `currentStart = i`.

```
int timDoanConTangDaiNhat(int a[], int n, int &k) {
    if (n == 0) {
        k = -1;
        return 0;
    }

    int maxLen = 1;
    int currentLen = 1;
    int maxStart = 0;
    int currentStart = 0;

    for (int i = 1; i < n; i++) {
        if (a[i] > a[i - 1])
            currentLen++;
        else {
            // Đoạn tăng bị ngắt, so sánh
            if (currentLen > maxLen) {
                maxLen = currentLen;
                maxStart = currentStart;
            }
            // Reset
            currentLen = 1;
            currentStart = i;
        }
    }

    // Kiểm tra lần cuối sau khi vòng lặp kết thúc
    if (currentLen > maxLen) {
        maxLen = currentLen;
        maxStart = currentStart;
    }

    k = maxStart;
    return maxLen;
}
```

Bài 43: Tìm đoạn con tổng lớn nhất

Sử dụng thuật toán Kadane. Dùng 2 biến: `maxSoFar` (tổng lớn nhất tìm được từ đầu đến giờ) và `maxEndingHere` (tổng lớn nhất của đoạn con kết thúc tại vị trí i).

Khi duyệt mảng, `maxEndingHere` sẽ được cập nhật bằng `maxEndingHere + a[i]`. Nếu `maxEndingHere < 0`, ta reset nó về 0 (vì một đoạn con âm sẽ không bao giờ làm tăng tổng). `maxSoFar` là giá trị lớn nhất của `maxEndingHere` trong suốt quá trình duyệt.

```
// Hàm hỗ trợ tìm max của 2 số
int max(int a, int b) {
    return (a > b) ? a : b;
}

int timDoanConTongLonNhat(int a[], int n) {
    if (n == 0) return 0;
```

```
int maxSoFar = a[0];
int maxEndingHere = a[0];

for (int i = 1; i < n; i++) {
    // Tổng của đoạn con kết thúc tại i là:
    // Hoặc là chính a[i] (nếu đoạn con trước đó âm)
    // Hoặc là a[i] + tổng đoạn con kết thúc tại i-1
    maxEndingHere = max(a[i], maxEndingHere + a[i]);

    // Cập nhật tổng lớn nhất tìm được
    maxSoFar = max(maxSoFar, maxEndingHere);
}

return maxSoFar;
}
```

Bài 44: Dịch phải xoay vòng K đơn vị

Dùng một mảng tạm `temp` kích thước n .

1. Giảm k bằng cách $k = k \% n$ (vì dịch n lần sẽ về như cũ).
2. Sao chép k phần tử cuối của mảng a (từ $a[n-k]$ đến $a[n-1]$) vào k vị trí đầu của mảng `temp` (từ `temp[0]` đến `temp[k-1]`).
3. Sao chép $n - k$ phần tử đầu của mảng a (từ $a[0]$ đến $a[n-k-1]$) vào các vị trí còn lại của `temp` (từ `temp[k]` đến `temp[n-1]`).
4. Sao chép toàn bộ mảng `temp` ngược lại vào mảng a .

```
void dichPhaiXoayVongKDonVi(int a[], int n, int k) {
    if (n == 0) return;

    k = k % n; // Chuẩn hóa k
    if (k == 0) return;

    int temp[n];

    // 1. Chép k phần tử cuối a vào đầu temp
    for (int i = 0; i < k; i++)
        temp[i] = a[n - k + i];
    // 2. Chép n-k phần tử đầu a vào cuối temp
    for (int i = 0; i < n - k; i++)
        temp[k + i] = a[i];
    // 3. Chép temp về a
    for (int i = 0; i < n; i++)
        a[i] = temp[i];
}
```

Bài 45: Tìm cặp số có tổng bằng X

Sử dụng 2 vòng lặp lồng nhau. Vòng lặp ngoài i chạy từ 0 đến $n-2$. Vòng lặp trong j chạy từ $i+1$ đến $n-1$. Kiểm tra nếu $a[i] + a[j] == X$ thì in ra.

```
void timCapSoTongBangX(int a[], int n, int X) {
    int timThay = 0;
    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++) {
            if (a[i] + a[j] == X) {
                printf("Tim thay cap: (%d, %d)\n", a[i], a[j]);
                timThay = 1;
            }
        }
    }
    if (timThay == 0)
        printf("Khong tim thay cap nao co tong bang %d\n", X);
}
```

Bài 46: Sắp xếp số lẻ tăng dần, số chẵn giữ nguyên

1. Tạo một mảng tạm `mangLe` chỉ để chứa các số lẻ.
2. Duyệt mảng `a`, chép tất cả các số lẻ vào `mangLe`.
3. Sử dụng hàm `sapXepTangDan` để sắp xếp `mangLe`.
4. Duyệt lại mảng `a`. Khi gặp một số lẻ, thay thế nó bằng phần tử tiếp theo trong `mangLe` đã sắp xếp.

```
void sapXepSoLeTangDan(int a[], int n) {
    int mangLe[n];
    int nLe = 0;

    // 1. Tách các số lẻ ra mảng phụ
    for (int i = 0; i < n; i++) {
        if (a[i] % 2 != 0) {
            mangLe[nLe] = a[i];
            nLe++;
        }
    }

    // 2. Sắp xếp mảng phụ (chứa số lẻ)
    sapXepTangDan(mangLe, nLe);

    // 3. Đưa các số lẻ đã sắp xếp về lại mảng chính
    int chiSoLe = 0;
    for (int i = 0; i < n; i++) {
        if (a[i] % 2 != 0) {
            a[i] = mangLe[chiSoLe];
            chiSoLe++;
        }
    }
}
```

Bài 47: Tìm phần tử còn thiếu

Tổng của n số từ 1 đến n là $S = n \times (n + 1) / 2$.

Tính tổng thực tế sum của n phần tử trong mảng a .

Phần tử bị thiếu chính là S - sum.

```
int timPhanTuConThieu(int a[], int n) {
    // Tổng lý thuyết của n+1 phần tử (từ 0 đến n)
    // Lưu ý: n là số phần tử, nhưng mảng chứa các số trong [0, n]
    // Ví dụ: n=4, mảng có 4 số trong [0, 4]
    long long tongLyThuyet = (long long)n * (n + 1) / 2;

    // Tổng thực tế của n phần tử trong mảng
    long long tongThucTe = 0;
    for (int i = 0; i < n; i++) {
        tongThucTe += a[i];
    }

    return (int)(tongLyThuyet - tongThucTe);
}

/*
// Ví dụ hàm main để kiểm tra Bài 47
int main() {
    int a[] = {0, 1, 3, 4}; // Mảng có n=4 phần tử
    int n = 4;
    // Các số đầy đủ là [0, 1, 2, 3, 4], thiếu số 2

    int soThieu = timPhanTuConThieu(a, n);
    printf("Số còn thiếu là: %d\n", soThieu); // Kết quả: 2

    return 0;
}
*/
```